1.6 Importing/exporting data

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 14 October 2026, 20:31

This handout provides practical guidance on importing, exporting and storing data in R using CSV, Excel and RDS formats, with strategies for troubleshooting common issues.

Table of contents

Recommended reading . . . 1
Preparation . . . 1
Where does R look for files? . . . 2
Exporting data . . . 2
Importing data . . . 4
A convenient alternative: RDS files . . . 6
Exercises . . . 7
References . . . 8

Script

You can find the full R script associated with this unit here.

Recommended reading

Winter (2020): Chapter 1.11

Preparation

The first section of an R script should always load the packages that are needed to run the code that follows. In this unit, we will need `readxl` and `writexl` to import and export MS Excel files.

```
library(readxl)
library(writexl)
```

If you haven't installed them yet, the R console will display an error message. For instructions on how to install an R package, consult the unit on Libraries.

Where does R look for files?

Whenever we save or load a file without specifying a full path, R uses a default folder: the **working directory**. It is important to know that this is *not* necessarily the folder your R script is stored in. You can check your current working directory at any time:

```
getwd()
```

```
[1] "C:/Users/yourname/Documents"
```

> Setting the working directory
>
> There are two common ways of telling R where to look:
>
> - **RStudio Projects (recommended)**: Go to `File` > `New Project` > `New Directory` (or `Existing Directory`) and choose a folder for your project (e.g., a term paper, thesis or research article). Whenever you open the project (via the `.Rproj` file), the working directory is automatically set to that folder. Keep your scripts and data files in this folder, and R will find everything.
> - **`setwd()`**: Alternatively, you can set the working directory manually, e.g., `setwd("C:/Users/yourname/Documents/my_project")`. In RStudio, you can also go to `Session` > `Set Working Directory` > `To Source File Location`.
>
> If your data files are in a subfolder of your working directory (e.g., a folder called `data`), you can simply include the folder name in the file name: `"data/frequency_data.csv"`. An error message such as `cannot open file 'frequency_data.csv': No such file or directory` almost always means that R is looking in the wrong folder!

Exporting data

Assume we'd like to export our data frame with word frequencies to a local file on our system. Let's quickly recreate the data frame:

```
# Generate data frame
data <- data.frame(lemma = c("start", "enjoy", "begin", "help"),
                   frequency = c(418, 139, 337, 281))

# Print contents
print(data)
```

```
  lemma frequency
1 start       418
2 enjoy       139
3 begin       337
4  help       281
```

There are two common formats in which tabular data can be stored:

- as .**csv** files ('**c**omma-**s**eparated **v**alues'; a plain text format that can be opened by any spreadsheet software, e.g., LibreOffice Calc or MS Excel)
- as .**xls**/.**xlsx** files (Microsoft Excel)

i Export to CSV

To save our `data` data frame in .csv format, we can use the `write.csv()` function. The argument `row.names = FALSE` prevents R from adding the row numbers (1, 2, 3, …) as an extra column to the file:

```
write.csv(data, "frequency_data.csv", row.names = FALSE)
```

The file is now stored in your working directory. You can open this file …

- in **LibreOffice**
- in **Microsoft Excel** via `File` > `Import` > `CSV file` > Select the file > `Delimited` and then `Next` > `Comma` and `Next` > `General` and `Finish`.

Clearly, opening CSV files in MS Excel is quite cumbersome, which is why it's better to export the data directly as an Excel file.

CSV files and German Excel

If your computer uses German language settings, Excel expects CSV files to use semicolons (`;`) as separators and commas (`,`) as decimal marks (e.g., `3,14` instead of `3.14`). Opening a regular CSV file will then display all values crammed into a single column. The functions `write.csv2()` and `read.csv2()` use exactly these conventions:

```
write.csv2(data, "frequency_data_de.csv", row.names = FALSE)
```

Export to Excel

We use the `write_xlsx()` function provided by the `writexl` package.

```
write_xlsx(data, "frequency_data.xlsx")
```

The file is now stored in your working directory. You should now be able to open it in MS Excel without any issues.

Importing data

Let's read the two files back into R.

Import from CSV

To import the CSV file, we can use the `read.csv()` function:

```
imported_csv <- read.csv("frequency_data.csv")
print(imported_csv)
```

```
  lemma frequency
1 start       418
2 enjoy       139
3 begin       337
4  help       281
```

Help, there's a mysterious column X!

If a CSV file is exported without `row.names = FALSE`, `write.csv()` stores the row numbers in the file as an unnamed first column. When reading it back in, `read.csv()` turns this column into a variable called `X`:

```
write.csv(data, "frequency_data_rownames.csv") # row numbers
 ↪  are written to the file
read.csv("frequency_data_rownames.csv")
```

```
  X lemma frequency
1 1 start       418
2 2 enjoy       139
3 3 begin       337
4 4  help       281
```

The best solution is to fix the problem at the source, i.e., to export with `row.names = FALSE`. If you can't re-export the file (e.g., because you received it from someone else), you can tell `read.csv()` to use the first column as row names instead:

```
read.csv("frequency_data_rownames.csv", row.names = 1)
```

```
  lemma frequency
1 start       418
2 enjoy       139
3 begin       337
4  help       281
```

A note on file encodings and separators

When working with CSV files, you may encounter issues with character encodings and separators, especially when:

- working with files from different operating systems,
- dealing with text containing special characters (é, ü, ñ, etc.), or
- importing files created in different regions (e.g., European vs. US).

The most common parameters for `read.csv()` are:

```
# For files with special characters (recommended default)
data <- read.csv("myfile.csv", fileEncoding = "UTF-8")

# For older files from Windows systems
data <- read.csv("myfile.csv", fileEncoding = "latin1")

# For files using semicolons as separators and commas as
 ↪  decimal marks
data <- read.csv("myfile.csv", sep = ";", dec = ",")

# ... which is equivalent to:
data <- read.csv2("myfile.csv")
```

- If you see garbled text like Ã© instead of é, try specifying `fileEncoding = "UTF-8"`.
- If your data appears in a single column, check whether your file uses semicolons (`;`) instead of commas (`,`) as separators.
- If numeric values are incorrect, verify whether the file uses commas or full stops as decimal marks.

Import from Excel

For importing the Excel file, we'll use the `read_xlsx()` function from the `readxl` package:

```
imported_excel <- read_xlsx("frequency_data.xlsx")
print(imported_excel)
```

```
# A tibble: 4 x 2
  lemma frequency
  <chr>     <dbl>
1 start       418
2 enjoy       139
3 begin       337
4 help        281
```

That's it! However, always check your imported data to make sure it has been read in correctly, especially when working with CSV files.

A convenient alternative: RDS files

If the main goal is to save an intermediate result and make it available for later use, the most efficient solution is to save the object to a local R data file ending in `.RDS`. RDS files have two major advantages:

- They store R objects **exactly as they are**. When data passes through CSV or Excel files, R has to guess the data type of each column when reading it back in (e.g., whether `418` is a number or a string), and it doesn't always guess correctly. An RDS file, by contrast, preserves all data types, column names and other properties of the object.
- They store data in compressed form, similar to .zip files, which keeps even large datasets (e.g., entire corpora) small on your hard drive.

In practice, we use the `saveRDS()` function and supply it with …

- … an R object (e.g., a vector, a data frame, a matrix, a plot, a statistical model – anything goes!) as well as
- … the desired name of the file.

```
# Save data frame "data" to the file "frequency_data.RDS"
saveRDS(data, "frequency_data.RDS")
```

To read a file back in, we need to indicate the file name (or the path relative to the working directory if the file is located in a subfolder, e.g., `"data/frequency_data.RDS"`).

```r
# Read in "frequency_data.RDS" and assign the contents to "data2"
data2 <- readRDS("frequency_data.RDS")

# Verify contents
print(data2)
```

```
  lemma frequency
1 start       418
2 enjoy       139
3 begin       337
4  help       281
```

Exercises

> **Solutions**
>
> You can find the solutions to the exercises here.

Exercise 1. Download the file `SCOPE_reduced.RDS` from this repository and read it into a variable named `SCOPE`. It contains data from the South Carolina Psycholinguistic Metabase (Gao, Shinkareva, and Desai 2022), specifically:

- Number of meanings (`Nsenses_WordNet`)
- Emotional valence ratings, which describe the pleasantness of a lexical stimulus on a scale from 1 to 9 (`Valence_Warr`)
- Data for nearly 200,000 words

Exercise 2. Using this database, retrieve:

1. the number of meanings for the verbs *start*, *enjoy*, *begin*, *help*. Store them in a data frame named `senses_df`.
2. emotional valence ratings for the words *fun*, *love*, *vacation*, *war*, *politics*, *failure*, *table*. Store them in a data frame named `valence_df`.

What do you notice about the valence ratings? Do they align with your intuitions about these words' emotional content?

> **Tip**
>
> This task is very similar to Exercise 3 in 1.4 Data frames!

Exercise 3. Export both `senses_df` and `valence_df` as .csv and .xlsx files, and read them back into R.

Exercise 4. Separators determine where one column ends and the next one begins in a CSV file. Investigate what happens when you read in `frequency_data.csv` with different separator settings:

```
# Comma separator (default)
imported_csv1 <- read.csv("frequency_data.csv", sep = ",")
print(imported_csv1)

# Tab separator
imported_csv2 <- read.csv("frequency_data.csv", sep = "\t")
print(imported_csv2)

# Semicolon separator
imported_csv3 <- read.csv("frequency_data.csv", sep = ";")
print(imported_csv3)
```

References

Gao, Chuanji, Svetlana V. Shinkareva, and Rutvik H. Desai. 2022. "SCOPE: The South Carolina Psycholinguistic Metabase." *Behavior Research Methods* 55 (6): 2853–84. https://doi.org/10.3758/s13428-022-01934-0.

Winter, Bodo. 2020. *Statistics for Linguists: An Introduction Using r*. New York; London: Routledge.